1. Trang bìa & Thông tin nhóm

TRƯỜNG CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC PHENIKAA

BÁO CÁO ĐỒ ÁN MÔN HỌC

CSE703153 - Kỹ thuật Lập trình An toàn

Kiểm thử white-box fuzzing trên simulator C và đối chiếu API tồn kho của Web Badminton

Thông tin	Nội dung bổ sung
Tên đề tài đăng ký	
Số nhóm / Lớp	
Giảng viên hướng dẫn	
Họ tên - MSSV thành viên 1	
Họ tên - MSSV thành viên 2	
Họ tên - MSSV thành viên 3	
Họ tên - MSSV thành viên 4	

Thông tin hành chính và phân công thực tế được để trống theo yêu cầu của nhóm.

Hà Nội, tháng ..... năm .....

MỤC LỤC

2. Tổng quan đề tài 3

3. Phương pháp luận 4

4. Phân tích lỗ hổng & Bản vá 5

5. Bảng tổng hợp before/after 7

6. Bằng chứng kiểm chứng 8

7. Phản biện & Bài học 12

8. Phụ lục 13

Trang bìa và thông tin nhóm: trang 1. Các phần 2-8 giữ đúng tên và thứ tự của mẫu báo cáo.

Bằng chứng số liệu được lấy từ file kết quả, không suy diễn từ số lượng file test. Log đầy đủ và mã nguồn liên quan được đính kèm trong PDF dưới dạng bang_chung_fuzzing.zip.

2. Tổng quan đề tài

2.1. Bài toán và sản phẩm làm cơ sở

Web Badminton là ứng dụng bán hàng có frontend React và backend Java/Spring Boot. Phần đồ án này chỉ chọn nghiệp vụ tồn kho của đơn hàng để mô hình hóa và kiểm thử: giữ số lượng khi đặt hàng, hoàn khi hủy và tất toán giữ chỗ khi giao đơn COD. Mã web không được chuyển sang C. C là ngôn ngữ của simulator kiểm thử riêng.

Thành phần	Hiện trạng được đối chiếu
Frontend	React 18.3.1, Vite; không sửa và không kiểm thử UI trong chiến dịch này.
Backend	Java 17, Spring Boot 3.2.5; Controller -> Service -> Repository.
Cơ sở dữ liệu	Driver MySQL trong backend; máy thực nghiệm trả MariaDB 10.4.32 của XAMPP. Không gọi đây là thử nghiệm MySQL 8.
Module chính	OrderService và ProductVariantService; dữ liệu product, variant và order.
Bộ kiểm thử	Simulator C11; generator và runner Python; GCC/gcov 16.2.0; JaCoCo 0.8.12.

2.2. Kiến trúc và ranh giới thay đổi

Luồng	Chuỗi xử lý đã triển khai
Ứng dụng web	React -> API Spring -> OrderService / ProductVariantService -> database của web. Không sửa mã trong phạm vi đồ án fuzz C.
Kiểm thử C	Mã service + đối chiếu log -> stock_spec.json -> bộ sinh riêng -> C sinh ra -> GCC -> hai chế độ fuzz -> gcov / oracle.
Công cụ của thầy	teacher_tables trong JSON -> badminton_spec.xlsx -> Spec Verification Lab -> CTL, static C, SMT và phân loại.
Đối chiếu backend	api_fuzz.py -> backend thử 127.0.0.1:18080 -> database fuzz_stock_ riêng -> so HTTP, C và tồn kho -> JaCoCo.

2.3. Phạm vi đã làm và chưa làm

Mô hình gồm một sản phẩm và tối đa một biến thể. Khi dùng biến thể, kho tổng ban đầu bằng kho biến thể. Các đại lượng chính là kho khả dụng a, kho giữ r, số lượng q và trạng thái. Bộ kiểm thử API dùng dữ liệu giả lập, chỉ đơn COD; không gọi PayOS thật hay sửa dữ liệu web.

Ngoài phạm vi: giao diện, voucher, giá, luồng thanh toán PayOS, nhiều biến thể, nhiều sản phẩm và tranh chấp đồng thời. Không có bản v0/v1 chứng minh lỗ hổng mới đã được vá trong lượt công việc này. Hai biểu thức được GUI gắn CWE-190 là đối tượng phân tích nguy cơ, không được coi là hai lỗi web đã xác nhận.

3. Phương pháp luận

3.1. Quy trình năm bước và nguồn dữ liệu

Bước	Thực hiện trong dự án
1. Xác định module	Đọc nghiệp vụ và chốt SHA-256 hai service. Log tạo đơn COD hỗ trợ đối chiếu, không tự suy ra đặc tả từ log.
2. Đặc tả và yêu cầu	JSON chứa reserve/settle và 6 yêu cầu truy vết; Excel 10 sheet là đầu vào của ứng dụng thầy.
3. Sinh và kiểm tra C	Dịch biểu thức bằng AST giới hạn; sinh hai hàm nghiệp vụ, tái sử dụng adapter C. Kiểm tra bằng oracle và ca hồi quy.
4. Kiểm thử động	Black-box và coverage-guided cùng target, cùng seed và giới hạn; sau đó gọi API backend thật trong môi trường riêng.
5. Phân loại và đối chiếu	Tổng hợp log, coverage, cảnh báo GUI, đối chiếu mã nguồn; không gán CVE cho dự án khi chưa có bằng chứng.

Chuỗi thầy nhận xét “Module -> log -> đặc tả -> req -> sinh code C simulator -> fuzzing -> output CWE/CVE” được đối chiếu với sản phẩm ở Phần 8. Chưa có công cụ trích xuất yêu cầu tự động từ log; đặc tả được xây qua việc đọc source, sau đó kiểm tra lại bằng log và hành vi API.

3.2. Công cụ và cấu hình có bằng chứng

```
gcc -std=c11 -Wall -Wextra -Werror -O0 --coverage
    -c stock_simulator.c -o stock_simulator.o
gcc --coverage stock_simulator.o -o stock_simulator.exe
gcov -b -c stock_simulator.c
```

Không sử dụng kết quả ASan, UBSan, TSan, cppcheck hoặc clang-tidy để đưa ra kết luận trong báo cáo vì chiến dịch này không chạy các công cụ đó. Static checker heuristic là thành phần riêng của GUI. Z3 được dùng trong ứng dụng của thầy, không phải một SMT solver do runner C triển khai.

3.3. Dữ liệu, oracle và tiêu chí dừng

Envelope C dài 28 byte: BDMN + 5 số int32 little-endian + uint32 action. Đây không phải JSON của API. Bit 3 chọn biến thể, bit 2 yêu cầu thao tác kết thúc lặp; hai bit thấp chọn giữ, hủy hoặc tất toán. Structured mode chọn biên 0, 1, 2, 99, 100, 101 và các giá trị sát INT_MAX. Black-box chọn độc lập; coverage-guided đột biến một trường từ corpus và giữ đầu vào làm tăng coverage.

Mỗi chế độ có 500 lượt tối đa, giới hạn 60 giây, seed 1/7/42. Mỗi tiến trình mục tiêu timeout 2 giây. Oracle Python viết độc lập so trạng thái và số lượng; bất biến kiểm tra kho không âm. Crash, timeout, sai bất biến hoặc lệch oracle làm dừng chiến dịch và lưu đầu vào bất thường. Ngân sách hết thời gian khác với một input bị timeout.

Nhánh mục tiêu dùng để so sánh là accepted và state=2, tức hủy thành công; đây là nhánh nghiệp vụ, không phải nhánh gây crash. Coverage thu theo cả tiến trình, không chỉ hai hàm nghiệp vụ. Độ phủ là chỉ số quan sát, không phải chứng minh không còn lỗi.

4. Phân tích lỗ hổng & Bản vá

Phần này tuân theo năm mục của mẫu nhưng ghi rõ tình trạng bằng chứng. Không có bản vá web mới được tạo trong chiến dịch. Hai mục dưới cùng thuộc CWE-190 và là hai biểu thức GUI cảnh báo [3], không phải hai lỗ hổng sản phẩm đã được fuzz tái hiện. Quy tắc INT32-C được dùng làm tham chiếu cho phép tính số nguyên có dấu trong C [4].

4.1. CWE-190 tại biểu thức tăng kho giữ chỗ

4.1.1. Vị trí & mô tả

Trong GUI: reserved_sum_expression và kiểm tra reserved_add, biểu thức r+q. Trong web: OrderService.validateProductForOrder, dòng 187-190; ProductVariantService.reserve, dòng 138-143. Phiên bản được mô tả là snapshot hiện tại ngày 15/10/2026, không phải v0/v1.

Mã Java hiện có: C:/Project/Web_badminton/backend/src/main/java/com/sports/service/OrderService.java, dòng 187-190.

```
187          long reservedStock = (long) product.getReservedStock() + quantity;
188          if (reservedStock > Integer.MAX_VALUE) {
189              throw new BadRequestException("Tồn kho giữ chỗ vượt giới hạn cho phép");
190          }
```

4.1.2. Nguyên nhân gốc rễ

Phép cộng int32 không kiểm soát có thể vượt 2.147.483.647. Hai hàm arithmetic GUI sinh chỉ return a+b và không chứa điều kiện nghiệp vụ. Với C, tràn số nguyên có dấu không được coi là phép wrap an toàn [4]. Trong Java hiện tại, phép cộng kho giữ được nâng lên long rồi kiểm tra giới hạn trước khi cập nhật.

4.1.3. Bằng chứng lỗi tồn tại

Log GUI do người thực hiện cung cấp ghi SAT với r=2147483647, q=1. Đây là phản ví dụ cho phép cộng không guard ở mức đặc tả số học. Mô hình overflow độc lập còn cho r=1,q=2147483647, nhưng q đó không hợp lệ với đơn hàng giới hạn 1-100. Không có log chứng minh đầu vào hợp lệ vượt guard của backend.

```
reserved_add: sat model={'r': 2147483647, 'q': 1}
reserved_add (int32 overflow?): sat model={'r': 1, 'q': 2147483647}
```

4.1.4. Cách vá

Không thực hiện bản vá mới. Guard long của OrderService và Math.addExact của biến thể đã có sẵn. Bộ sinh C chuyển phép tính kiểm tra sang int64_t để giữ điều kiện tương ứng. Đây là mô hình hóa phòng vệ hiện có, không phải diff trước/sau cùng một lỗi. Chưa có cặp v0/v1 hợp lệ để trình bày bản vá.

4.1.5. Bằng chứng đã vá

Không ghi “đã vá”. Bằng chứng hiện có chỉ xác nhận kiểm tra phòng vệ trong phạm vi fixture: ca hồi quy 18 và 20 từ chối vượt INT_MAX (code=3), ca 19 chấp nhận sát giới hạn. API có fixture reserved=INT_MAX,q=1 bị từ chối và kho không đổi; fixture này được chèn trực tiếp vào DB thử.

4.2. CWE-190 tại biểu thức hoàn kho khả dụng

4.2.1. Vị trí & mô tả

Trong GUI: release_sum_expression và available_release, biểu thức a+q. Trong web: OrderService.settleReservedStock, dòng 296-307; ProductVariantService.settle, dòng 144-149. Phân tích chỉ áp dụng snapshot được kiểm tra hash, không chứng minh một bản trước đó có lỗi.

Mã hoàn kho hiện có: C:/Project/Web_badminton/backend/src/main/java/com/sports/service/OrderService.java, dòng 301-307.

```
301             if (product.getReservedStock() < item.getQuantity()) {
302                 throw new BadRequestException("Tồn kho giữ chỗ không khớp; cần kiểm tra
    trước khi xử lý đơn");
303             }
304             product.setReservedStock(product.getReservedStock() - item.getQuantity());
305             if (restoreAvailable) product.setStock(product.getStock() +
    item.getQuantity());
306             ProductVariantService.settle(item.getVariant(), item.getQuantity(),
    restoreAvailable);
307             ProductVariantService.refreshTotals(product);
```

4.2.2. Nguyên nhân gốc rễ

SMT kiểm tra phép cộng như biểu thức độc lập, không mang theo điều kiện "đã giữ cùng q trước khi hủy". Trên mô hình một sản phẩm không thay đổi bởi tác nhân khác, sau giữ a=a0-q; sau hủy a+q=a0. Vì vậy phản ví dụ a=INT_MAX,q=1 của phép cộng độc lập không tự chứng minh chuỗi reserve-cancel của web bị tràn.

4.2.3. Bằng chứng lỗi tồn tại

```
available_release: sat model={'a': 2147483647, 'q': 1}
available_release (int32 overflow?): sat model={'a': -1, 'q': -2147483648}
```

Mô hình thứ hai chứa kho âm và số lượng âm, ngoài miền fixture và API đã đặc tả. Không có đầu vào HTTP hợp lệ được ghi nhận gây lỗi hoàn kho trong lượt chạy đạt. Không thay một cảnh báo biểu thức thành kết luận có thể khai thác trên website.

4.2.4. Cách vá

Không sửa mã hoàn kho của web. ProductVariantService đã có Math.addExact; phần product vẫn dùng phép cộng int như source hiện tại. Adapter C dùng java_add để mô phỏng wrap Java khi cần, không dựa vào tràn signed C. Việc này mô tả hành vi hiện hữu, không được gọi là bản vá backend.

4.2.5. Bằng chứng đã vá

Không có bằng chứng "sau vá" vì chưa có bản vá. Ca C 4 và 9 cho giữ 2, hủy rồi hủy lại: available=10, reserved=0, state=2. API cũng kiểm tra hủy lặp không hoàn lần hai. Kết quả này chỉ xác nhận các ca đã chạy, không loại trừ thay đổi kho đồng thời hoặc fixture nhiều biến thể.

4.3. Giới hạn phân loại CWE/CVE

CWE mô tả lớp điểm yếu, còn CVE là mã một lỗ hổng cụ thể đã được ghi nhận. CVE-2016-5195 được ứng dụng đưa ra làm ví dụ là Dirty COW của Linux, liên quan race condition [5]; không phải CVE của Web Badminton và không được dùng để xác nhận phép cộng kho. Báo cáo không ghi số CVE cho dự án và không tạo lỗi bộ nhớ giả để có finding.

5. Bảng tổng hợp before/after

Giữ cấu trúc bảng của mẫu. Cột v0/v1 được ghi theo tình trạng bằng chứng thật: không có cặp bản nguồn trước/sau được chứng minh cho lỗ hổng mới. Không thay bằng hai đoạn code của hai chương trình khác nhau để nhận là một bản vá.

CWE / vị trí	Trạng thái v0	Trạng thái v1 / hiện tại	Công cụ xác minh
CWE-190; reserved_add / tăng reserved	Không có v0 web được xác minh. GUI SAT trên r+q không guard: INT_MAX+1.	Không tạo v1. Backend hiện có guard long/Math.addExact; C từ chối ca 18/20 và API từ chối fixture tương ứng.	GUI SMT; regression.json; API summary.json; đọc source.
CWE-190; available_release / hoàn available	Không có v0 web được xác minh. GUI SAT trên a+q độc lập, không mang điều kiện đã reserve.	Không tạo v1. C và API qua các ca hoàn kho/lặp; chưa xác nhận lỗi có thể khai thác hoặc đã vá.	GUI SMT; ca hồi quy 4/9; API hủy lặp và kiểm tra DB.

5.1. Diễn giải trước/sau đúng với sản phẩm

Trước khi bổ sung bộ sinh, simulator được viết tay và fuzz riêng. Sau khi bổ sung, reserve/settle được sinh từ JSON, có kiểm tra đổi đặc tả đổi hành vi, snapshot và đối chiếu API. Đây là thay đổi hạ tầng kiểm thử, không phải before/after loại bỏ một CWE của website.

Tiêu chí	Bằng chứng hiện có	Không được kết luận
Sinh mã	9 kiểm tra bộ sinh; sinh từ quy tắc JSON, adapter tái sử dụng.	Toàn bộ mã C được GUI sinh hoặc mọi dòng adapter đến từ JSON.
Kiểm thử C	25 ca hồi quy và chiến dịch đủ 3.000 lượt.	Có lỗi v0 và đã vá hết, hoặc đã phủ mọi nhánh.
Kiểm thử web	90 ca API + 8 ca validation trên server/database thử.	Đã kiểm thử frontend, PayOS hoặc toàn bộ website.
Phân loại	Hai cảnh báo biểu thức CWE-190 trong GUI.	Hai lỗ hổng backend đã tái hiện hay CVE mới.

6. Bằng chứng kiểm chứng

6.1. Tính toàn vẹn và kết quả bộ sinh

Báo cáo kiểm tra lại SHA-256 source hiện tại với metadata của chiến dịch. Các snapshot source C, JSON, bộ sinh và runner trong chiến dịch chính khớp hash đã lưu. Chín kiểm tra bộ sinh được chạy lại khi dựng báo cáo; stdout/stderr đầy đủ được đính kèm.

```
test_backend_change_stops_generation
    (__main__.GeneratorTests.test_backend_change_stops_generation) ... ok
test_backend_fingerprint (__main__.GeneratorTests.test_backend_fingerprint) ... ok
test_generated_regressions (__main__.GeneratorTests.test_generated_regressions) ... ok
test_generation_is_deterministic (__main__.GeneratorTests.test_generation_is_deterministic) ...
     ok
test_handwritten_business_rules_are_not_copied
    (__main__.GeneratorTests.test_handwritten_business_rules_are_not_copied) ... ok
test_operation_sets_are_present (__main__.GeneratorTests.test_operation_sets_are_present) ...
    ok
test_reject_arbitrary_code (__main__.GeneratorTests.test_reject_arbitrary_code) ... ok
test_requirement_traceability (__main__.GeneratorTests.test_requirement_traceability) ... ok
test_spec_changes_executed_behavior
    (__main__.GeneratorTests.test_spec_changes_executed_behavior) ... ok

----------------------------------------------------------------------
Ran 9 tests in 1.055s

OK
```

Kiểm tra thay giới hạn q từ 100 xuống 50 làm C từ chối q=51 trong khi oracle gốc chấp nhận. Kiểm tra sửa quy tắc nghiệp vụ viết tay nhưng không sửa JSON không làm mã sinh đổi; điều đó hỗ trợ kết luận hai hàm nghiệp vụ không được sao chép từ bản viết tay.

6.2. So sánh hai chế độ trên cùng simulator

Seed	Chế độ	Lượt	Tổng giây	Lượt tới đích	Giây tới đích
1	Black-box	500	16.788	13	0.416
1	Coverage-guided	500	18.705	143	4.606
7	Black-box	500	17.309	7	0.251
7	Coverage-guided	500	20.645	92	2.974
42	Black-box	500	18.430	26	1.151
42	Coverage-guided	500	17.386	142	4.575

Bảng 1. Nguồn: summary.json của chiến dịch 20261015-124739-faca951a; đích là hủy thành công (accepted, state=2), không phải crash. Thời gian là đo thực tế của runner, gồm khởi chạy tiến trình và gcov.

Ở cả ba seed, black-box tới đích sớm hơn. Coverage cuối của hai chế độ bằng nhau. Với mục tiêu nhỏ và tập biên hạn chế này, không có cơ sở tuyên bố coverage-guided tốt hơn. Ba seed chưa phải kiểm định thống kê về mọi chương trình hoặc mọi cấu hình máy.

6.3. Độ phủ và bất thường C

Chỉ số	Kết quả chiến dịch chính
Hồi quy	25/25 ca khớp oracle; tách biệt với 3.000 lượt fuzz.
Fuzz	6 chiến dịch x 500 lượt = 3.000 lượt.
Dòng	Mỗi chiến dịch 62/63 dòng = 98,41%.
Nhánh	Mỗi chiến dịch 79/104 nhánh = 75,96%; không cộng trùng giữa các seed.
Bất thường	0 crash, 0 timeout mục tiêu, 0 sai bất biến, 0 lệch oracle.

Dòng invalid_fixture chưa được structured fuzz của chiến dịch chính chạm tới, nhưng ca hồi quy 25 đã kiểm tra fixture âm. Coverage fuzz ở bảng không trộn coverage hồi quy. Với fixture một biến thể có a=va,r=vr, một số nhánh bảo vệ biến thể bị điều kiện kho tổng chặn trước. Vì vậy TC-6 chưa chứng minh nhánh thiếu kho biến thể độc lập; không ghi đầy đủ branch coverage.

Chiến dịch bổ sung 20261015-124958-507184fc chạy 2.861 lượt, có một chế độ chạm trần 60 giây ở lượt 361. Source C có cùng SHA-256 với chiến dịch chính. Không gộp thành 3.000 lượt, không dùng để suy ra thời gian công bằng khi tải hệ thống khác nhau. Log đầy đủ của cả hai chiến dịch nằm trong phụ lục điện tử.

6.4. Kiểm thử API và trạng thái database

Kết quả dưới đây đến từ summary.json của api-fuzz_stock_0aa67d3960ad, trạng thái pass. Server thử 127.0.0.1:18080, database fuzz_stock_0aa67d3960ad; dữ liệu giả lập, đơn COD. Backend thử được biên dịch javac -parameters --release 17 vào thư mục riêng, dùng JAR phụ thuộc đã có. Không dùng trực tiếp các file .class cũ của IDE và không sửa mã backend.

Nhóm kết quả tạo đơn	Số ca	Kiểm tra tiếp theo
HTTP 201	22	Replay cùng ID; xung đột khóa trả 409; chặn người khác hủy; kết thúc và lặp không đổi kho.
HTTP 400	37	Đầu vào bị từ chối theo điều kiện; kiểm tra kho tổng/biến thể không đổi.
HTTP 409	31	Thiếu kho; kiểm tra trạng thái tồn khớp simulator.

Bảng 2. Tổng 90 ca gồm 30 ca biên cố định và 60 ca sinh với 3 seed, mỗi seed 20 ca. 22 ca tạo đơn thành công. Đây là số ca, không phải tổng số request HTTP vì mỗi ca có thể gọi nhiều request.

Ca validation / xác thực	HTTP thực tế	Kho không đổi
missing-jwt	401	Có
invalid-key	400	Có
missing-key	400	Có
quantity-type-0	400	Có
quantity-type-1	400	Có
quantity-type-2	400	Có
quantity-type-3	400	Có
empty-items	400	Có

Bảng 3. Tám ca bổ sung: thiếu JWT; khóa idempotency sai/thiếu; quantity null, phân số, chuỗi không hợp lệ, vượt int32; danh sách items rỗng. Tách riêng khỏi 90 ca đối chiếu mô hình.

6.5. Coverage Java do HTTP thực thi

Lớp	Dòng	Nhánh
OrderService	211/296 (71.28%)	84/204 (41.18%)
ProductVariantService	25/183 (13.66%)	22/234 (9.40%)

Bảng 4. Nguồn JaCoCo coverage.xml và summary.json từ chính backend thử. Tỷ lệ là trên toàn lớp, không riêng các hàm tồn kho. Phần lớn ProductVariantService là quản trị biến thể ngoài phạm vi này. Không coi độ phủ đó là 100% module hoặc toàn web.

API runner sinh dữ liệu theo biên và seed, không dùng JaCoCo để quyết định đột biến. Vì vậy đây là kiểm thử HTTP có đo coverage và đối chiếu trạng thái, không phải coverage-guided HTTP. JaCoCo ghi nhận hoạt động thực thi; cơ chế dẫn hướng white-box nằm ở runner C.

6.6. Kết quả ứng dụng Spec Verification Lab

Nguồn: toàn bộ nội dung pipeline và mã C do người thực hiện gửi sau khi nạp Excel. Đây là xác nhận chạy thủ công, không phải ảnh GUI do công cụ tự chụp. Nội dung gốc được lưu trong gui_pipeline_nguoi_dung.txt và đính kèm. Không dựng ảnh màn hình giả từ văn bản.

Thành phần	Quan sát	Ý nghĩa đúng
Nạp đặc tả	Tên badminton_stock; liệt kê 6 test case.	Xác nhận đọc và xử lý đặc tả; TestCase là tài liệu truy vết.
Chương 1	Không deadlock; 5/5 CTL đúng; mutual exclusion OK.	Chỉ trên đồ thị trừu tượng có 5 trạng thái; AG(!violation) đúng tầm thường vì không có nhãn violation.
Chương 2	Không phát hiện lỗi heuristic.	Chỉ hai hàm arithmetic; không quét toàn bộ C simulator hoặc Java.
Chương 3	Hai biểu thức được báo SAT và gắn CWE-190.	Kiểm tra số học độc lập chưa đủ để xác nhận input web hợp lệ gây lỗi.
Chương 4	Trống, không có số lượt fuzz.	GUI không chạy fuzz C trong pipeline được quan sát. Không chuyển số liệu runner C sang ô trống này.
Mã sinh	Hai hàm return a+b.	Không chứa guard số lượng, thiếu kho, replay hay trạng thái của simulator đầy đủ.

6.7. Hai nguồn sinh C khác nhau

Bộ sinh riêng của nhóm sinh reserve/settle từ JSON và ghép adapter C, sau đó runner biên dịch và chạy. Ứng dụng của thầy sinh hai hàm arithmetic từ các dòng HamSinhMa trong Excel. Hai nguồn này cùng phục vụ đặc tả tồn kho nhưng không phải cùng một artifact hay một pipeline tự động liên tục.

```
/* Mã ứng dụng thầy sinh, theo kết quả người dùng cung cấp */
int reserved_sum_expression(int a, int b) {
    return a + b;
}
int release_sum_expression(int a, int b) {
    return a + b;
}
```

Caption: đoạn mã trên minh chứng giới hạn mã sinh của GUI, không được gọi là toàn bộ simulator C. Bản đầy đủ nguyên văn nằm trong log GUI đính kèm.

6.8. Tình trạng nghiệm thu kỹ thuật

Các bước triển khai 1-6 của phương án đã thống nhất có bằng chứng trong phạm vi đã chọn. Lượt chạy thủ công vừa được cung cấp bổ sung xác nhận còn thiếu ở bước 6. README hiện còn dòng “GUI chưa xác nhận lại”; đó là trạng thái trước khi nhận log này, không được dùng thay cho kết quả mới. Báo cáo không tự xác nhận thay giảng viên rằng phương án bộ sinh riêng và fuzz C bên ngoài đáp ứng mọi tiêu chí chấm.

7. Phản biện & Bài học

7.1. Nếu bắt đầu lại, nên thiết kế gì khác?

Nên chốt một đặc tả có miền dữ liệu, trạng thái và điều kiện bảo vệ ngay từ đầu để tránh việc Excel và simulator diễn giải khác nhau. Bộ sinh, oracle và source backend cần có liên kết phiên bản và test biến đổi đặc tả. Cần kiểm tra sớm khả năng thật của ứng dụng thầy thay vì giả định nó fuzz C chỉ vì có nút sinh mã C/Python.

7.2. Những rủi ro nào chưa đủ bằng chứng xác nhận?

Các phép cộng/tổng nhiều biến thể, thay đổi tồn kho đồng thời, lock và rollback của nhiều đơn chưa được mô hình hiện tại bao phủ. Những phần đó là khoảng trống kiểm thử, không phải lỗ hổng đã xác nhận. INT_MAX trong fixture thử không chứng minh quản trị hoặc người mua tạo được trạng thái tương ứng qua các API hợp lệ. Cũng chưa có chiến dịch sanitizer để kết luận về an toàn bộ nhớ C.

7.3. Bài học phương pháp luận

Cần phân biệt ba mức: cảnh báo biểu thức, lỗi simulator và lỗi web được tái hiện. SAT hoặc 0 crash không tự chuyển thành “web có lỗi” hay “web an toàn”. Khi hai chế độ có độ phủ bằng nhau và black-box đạt đích sớm hơn, phải báo đúng thay vì chọn số liệu có lợi cho white-box.

7.4. Những phần còn thiếu so với mẫu báo cáo và nhận xét

Khoảng trống	Ảnh hưởng
Chưa có lỗi web xác nhận và bản v0/v1	Chưa thể đáp ứng bằng chứng lỗi-bản vá trước/sau của mẫu theo nghĩa đầy đủ. Bảng Phần 5 ghi rõ, không giả lập bằng chứng.
Fuzz C không chạy trong GUI	Chỉ đạt phương án chạy runner riêng đã thống nhất; cần thầy chấp nhận cách tích hợp bằng chứng này.
Log chưa tự chuyển thành đặc tả	Quy tắc được đọc từ source và đối chiếu log; chưa tự động hóa toàn bộ chuỗi thầy ghi.
Coverage chưa đầy đủ	C 75,96% nhánh; nhánh biến thể độc lập, đồng thời và chức năng ngoài tồn kho chưa được chứng minh.
Thông tin nhóm và bản xuất GUI chính thức	Thông tin hành chính/phân công còn trống; chưa có PDF/DOCX nguyên bản xuất từ ứng dụng thầy trong hồ sơ hiện tại.

Kết luận: đã xây và kiểm chứng hệ thống thử nghiệm trong phạm vi nhỏ, có số liệu tái kiểm tra. Báo cáo này không tuyên bố đã vá toàn bộ lỗ hổng hoặc bảo đảm đạt điểm. Các mục thiếu được liệt kê để người thực hiện và giảng viên đánh giá, không tự sửa sản phẩm để làm đẹp kết quả.

8. Phụ lục

8.1. Hồ sơ nguồn và phân công thực tế

Họ tên	MSSV	Công việc thực tế	Tỷ lệ / xác nhận

Các ô trên để trống theo yêu cầu. Không suy đoán đóng góp của thành viên từ lịch sử hội thoại hoặc tên nhóm.

8.2. Hướng dẫn tái lập - không dùng dữ liệu web thật

Chạy PowerShell tại C:/Project/Web_badminton. Python cần thư viện chuẩn để chạy bộ fuzz. Trên máy thực nghiệm, Python bundled đã được dùng; không mặc định lệnh python của mọi máy đều trỏ đúng phiên bản.

```
python -B C:/Project/Web_badminton/software_security_fuzz/test_generator.py
python -B C:/Project/Web_badminton/software_security_fuzz/fuzz_runner.py --iterations 500
    --seconds 60 --seeds 1 7 42
```

Lệnh fuzz tự sinh C trước khi biên dịch. Không cần frontend, backend hay MySQL cho bước C. Đọc thư mục kết quả mới được in ở cuối lệnh; dùng summary.json để biết số lượt thực sự đã chạy.

```
python -B C:/Project/Web_badminton/software_security_fuzz/api_fuzz.py
    --run-isolated
    --simulator <THU_MUC_CHIEN_DICH>/regression/stock_simulator.exe
    --iterations 20
```

Lệnh trên được xuống dòng để đọc; khi chạy, ghép thành một dòng và thay đường dẫn mẫu bằng binary của chiến dịch đã PASS. Cần MySQL/MariaDB đang chạy và cổng 18080 rảnh. Mỗi lượt API tạo database fuzz_stock_ mới, giữ lại sau chạy; chỉ dữ liệu giả. Không trỏ runner sang server web hiện tại.

Excel: nạp C:/Project/Web_badminton/software_security_fuzz/badminton_spec.xlsx bằng “Nạp từ file Excel khác”, chọn “Chạy TOÀN BỘ Pipeline”. Excel không tự cập nhật từ kết quả fuzz. Chương 4 có thể trống trong cấu hình chỉ khai báo hàm C arithmetic; số liệu fuzz C lấy từ runner riêng.

8.3. Bản nguồn và phiên bản

C:/Project/Web_badminton/backend/src/main/java/com/sports/service/OrderService.java

`SHA-256: 7c40dc928498fcd79af6bb278f8ce4a0f4b759f1efe76b3af94a99ef9caea53d`

C:/Project/Web_badminton/backend/src/main/java/com/sports/service/ProductVariantService.java

`SHA-256: d19b7c8e5271a2cc005df62b172a773ddeb8655935cd0ff88a1e1c5a0648f6d7`

```
SHA-256 C sinh: a939d50d027da0a421272b0ca46374d25239cd5653e26f715d282fbf425e9362
SHA-256 JSON: 3970e36c1150c053390d9b6b471560f7c3a9c3c5488d9841848143c8c14cbcc7
```

Snapshot runner và generator của chiến dịch chính có hash riêng trong metadata.json; không giả định giống byte với công cụ hiện tại. Chiến dịch bổ sung lưu reference_stock_simulator.c và tái sinh đúng nội dung C. Nếu source backend đổi hash, cần đối chiếu đặc tả trước khi sử dụng lại số liệu.

8.4. Truy vết yêu cầu và hồi quy

Yêu cầu	Nội dung	Nguồn / bằng chứng
REQ-1 / TC-1	q từ 1 tới 100	OrderService dòng 114-125; hồi quy 11/12/15 từ chối, 13 chấp nhận.
REQ-2 / TC-2	Đủ kho mới giữ	OrderService dòng 178-185; ca 16 kho 1 chấp nhận, 17 kho 0 từ chối.
REQ-3 / TC-3	Reserved không tràn	OrderService dòng 187-190; ca 18/20 từ chối, 19 sát giới hạn.
REQ-4 / TC-4	Hủy hoàn đúng một lần	OrderService dòng 275-307; ca 4/9 và API hủy lặp.
REQ-5 / TC-5	Tất toán không trừ khả dụng lần hai	ProductVariantService dòng 144-149; ca 5/10, API giao COD lặp.
REQ-6 / TC-6	Từ chối thiếu kho biến thể	ProductVariantService dòng 138-142; ca 17 bị chặn tại kho tổng trước, chưa chứng minh nhánh riêng.

Bảng 5. Sáu test case trong Excel là tài liệu. Bằng chứng thực thi được trỏ riêng tới C và API, không coi việc GUI in danh sách là đã chạy test.

8.4.1. Toàn bộ kết quả hồi quy C

Ca	Trạng thái C	Code	a / r / state	Oracle
1	accepted	0	8/2/1	ok
2	accepted	0	10/0/2	ok
3	accepted	0	8/0/3	ok
4	accepted	0	10/0/2	ok
5	accepted	0	8/0/3	ok
6	accepted	0	8/2/1	ok
7	accepted	0	10/0/2	ok
8	accepted	0	8/0/3	ok
9	accepted	0	10/0/2	ok
10	accepted	0	8/0/3	ok
11	rejected	1	10/0/0	ok
12	rejected	1	10/0/0	ok
13	accepted	0	9/1/1	ok
14	rejected	2	10/0/0	ok
15	rejected	1	10/0/0	ok
16	accepted	0	0/1/1	ok
17	rejected	2	0/0/0	ok
18	rejected	3	100/2147483647/0	ok
19	accepted	0	0/2147483647/1	ok
20	rejected	3	100/2147483548/0	ok
21	accepted	0	0/100/1	ok
22	invalid_length	-	-/-/-	ok
23	invalid_length	-	-/-/-	ok
24	header_0	-	-/-/-	ok
25	invalid_fixture	-	-/-/-	ok

Bảng 6. Toàn bộ 25 ca hồi quy, tóm tắt trực tiếp từ regression.json. JSON đính kèm giữ đủ input hex, kết quả, log và số lượng biến thể; bảng không thay thế dữ liệu gốc.

8.5. Bằng chứng đầy đủ đính kèm

PDF chứa tệp đính kèm bang_chung_fuzzing.zip. Mở bằng trình đọc hỗ trợ Attachments để lưu gói bằng chứng. Mỗi file trong gói có SHA-256 trong manifest.json. Nếu trình xem PDF không hiển thị tệp đính kèm, dùng các đường dẫn tuyệt đối dưới đây trong mã nguồn nộp kèm.

Hồ sơ	Nội dung đầy đủ
chien_dich_chinh/	summary.json/csv, metadata, regression.json, sáu events.csv, compile.log, gcov và target-input.bin; snapshot C/JSON/generator/runner.
chien_dich_bo_sung/	summary.json/csv và metadata của lượt 2.861 input; sáu events.csv và snapshot source/adapter.
api/	summary.json chứa 90 ca và 8 validation; coverage.xml, api.exec, backend.log, build.log và log JaCoCo.
nguon_hien_tai/	Các file fuzz đang có và hai service Java dùng đối chiếu; không gồm secrets/config.
kiem_tra_bao_cao/	Log 9 kiểm tra generator và bản chép đầy đủ kết quả GUI người dùng cung cấp.

C:/Project/Web_badminton/backend/target/software-security-fuzz/20261015-124739-faca951a

C:/Project/Web_badminton/backend/target/software-security-fuzz/20261015-124958-507184fc

C:/Project/Web_badminton/backend/target/software-security-fuzz/api-fuzz_stock_0aa67d3960ad

C:/Project/Web_badminton/software_security_fuzz/badminton_spec.xlsx

Sáu file events.csv của chiến dịch chính có tổng 3.000 dòng dữ liệu, không tính header. Không cắt bớt các log này trong phụ lục điện tử. Không có log input lỗi mới vì chiến dịch đạt chưa ghi nhận input bất thường. Không đính kèm token, mật khẩu, .env hoặc dữ liệu web thật.

8.6. Nhật ký nghiệp vụ - trích đoạn có chú thích

```
2026-10-15T12:52:20.808+07:00  INFO 15844 --- [.1-18080-exec-4]
    com.sports.service.OrderService          : Tạo đơn ID=20, phương thức=COD, tổng
    tiền=130000.00
2026-10-15T12:52:22.495+07:00  INFO 15844 --- [.1-18080-exec-7]
    com.sports.service.OrderService          : Tạo đơn ID=21, phương thức=COD, tổng
    tiền=230000.00
2026-10-15T12:52:23.191+07:00  INFO 15844 --- [.1-18080-exec-1]
    com.sports.service.OrderService          : Tạo đơn ID=22, phương thức=COD, tổng
    tiền=130000.00
```

Caption: ba dòng cuối log tạo đơn COD của backend thử. Toàn bộ backend.log có trong gói; log này xác nhận luồng thử đi vào OrderService nhưng không tự chứng minh mọi bước kho đúng. Kết luận tồn kho lấy thêm từ các assert HTTP/DB được summary.json ghi nhận.

8.7. Log GUI nguyên văn

Bản chép đầy đủ gồm output pipeline, sáu test case và hai hàm C sinh ra, nằm trong kiem_tra_bao_cao/gui_pipeline_nguoi_dung.txt của tệp đính kèm. Nội dung được giữ theo người cung cấp, kể cả CVE minh họa chưa phù hợp; diễn giải và giới hạn nằm ở Phần 4 và Phần 6.

8.8. Đối chiếu cuối với sản phẩm và yêu cầu của thầy

Hạng mục	Đối chiếu hiện tại	Kết luận
Module	Tồn kho một sản phẩm, tối đa một biến thể.	Có, phạm vi hẹp và công khai.
Log	Log C và log backend thử; chưa trích đặc tả tự động từ log.	Có bằng chứng, chưa tự động toàn chuỗi.
Đặc tả / req	JSON executable và Excel 10 sheet, 6 yêu cầu truy vết.	Có; cần phân biệt tài liệu và test thực thi.
Sinh C simulator	Bộ sinh riêng sinh reserve/settle; adapter tái sử dụng.	Có theo phương án đã thống nhất; không phải GUI sinh toàn bộ.
White-box fuzzing	gcov-guided giữ corpus theo novelty; so với black-box.	Có, chưa concolic/SMT trong runner C.
Kiểm thử website	HTTP thật trên bản backend với database thử.	Có cho ca tồn kho/COD; không kiểm thử giao diện hoặc web sản xuất.
Output CWE/CVE	GUI có hai CWE-190 biểu thức; chưa có lỗi web/CVE xác nhận.	Không được báo hai lỗi web đã sửa.
Pipeline của thầy	Người thực hiện cung cấp output chạy thành công.	Đã xác nhận nạp Excel và pipeline; Chương 4 vẫn trống.
Mẫu 8 phần	Đủ thứ tự, bảng before/after trung thực, log đầy đủ đính kèm.	Đủ cấu trúc; chưa đủ bằng chứng v0/v1 lỗi-bản vá.

Danh sách còn thiếu, không tự sửa: thông tin nhóm/phân công; cặp lỗi và bản vá v0/v1 có bằng chứng nếu thầy yêu cầu bắt buộc; xác nhận chấp nhận fuzz C bên ngoài GUI; thêm bằng chứng xuất PDF/DOCX gốc của GUI nếu cần nộp; nhánh biến thể độc lập, đồng thời, sanitizer và các chức năng ngoài phạm vi. Những mục mở rộng trong hướng dẫn Excel (memory/liveness, lỗi mẫu bộ nhớ, target Python) không được tự thêm thành tính năng hay lỗi của web. Cần thầy xác nhận mục nào bắt buộc với nhóm.

Báo cáo chỉ bổ sung hồ sơ, không sửa code hoặc cấu hình web và không tạo/xóa database mới khi dựng. Các kết quả không bảo đảm tính an toàn tuyệt đối hoặc kết quả chấm điểm.

8.9. Tài liệu tham khảo

[1] report_guide.pdf. Hướng dẫn Viết Báo cáo Đồ án, CSE703153, năm học 2026-2027. Nguồn người thực hiện cung cấp: C:/Users/vxg09/Downloads/report_guide.pdf. Đọc toàn bộ 9 trang.

[2] Mã nguồn Web Badminton, đặc tả và artifact kiểm thử tại các đường dẫn Phần 8.5; kết quả pipeline do người thực hiện cung cấp trong hội thoại. Mốc đối chiếu 15/10/2026.

[3] MITRE. CWE-190: Integer Overflow or Wraparound. https://cwe.mitre.org/data/definitions/190.html. Tra cứu 15/10/2026; dùng để mô tả lớp nguy cơ, không xác nhận finding của web.

[4] SEI CERT C. INT32-C. Ensure that operations on signed integers do not result in overflow. https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/rules/integers-int/int32-c/. Tra cứu 15/10/2026.

[5] CVE Program. CVE-2016-5195. https://www.cve.org/CVERecord?id=CVE-2016-5195. Tra cứu 15/10/2026; chỉ dùng làm rõ CVE ví dụ của GUI thuộc Linux, không thuộc dự án.

[6] Tài liệu hướng dẫn đề tài, hướng dẫn Excel và bộ ATPM/ch4_dynamic do người thực hiện cung cấp trong quá trình làm bài. Chỉ những chức năng và kết quả đã đối chiếu mã/log mới được nhận là sản phẩm đã thực hiện; không nhận mọi tính năng của tài liệu mẫu là của website.